تمرین

تمرین سری اول – بخش تئوری

نگارنده
محمدحسین سبزعلیان

شمارهٔ دانشجویی
۴۰۲۱۰۱۹۰۶

درس
سیستم‌عامل

استاد
دکتر اسدی

راهنمای خواندن

در انتهای هر بخش «منبع» آمده است. عبارت «اسلاید» یعنی پاسخ بر پایهٔ اسلایدهای فصل ۱ (مقدمه) و فصل ۲ (ساختار سیستم‌عامل) نوشته شده است و عبارت «تکمیلی» یعنی آن مطلب در اسلایدها نبوده و پاسخ از دانش درسی (کتاب سیلبرشاتس، مستندات لینوکس و مقالات مربوط) تکمیل شده است.

۱ مجازی‌سازی

بخش آ – اجرای مستقیم چند سیستم‌عامل بدون لایهٔ مجازی‌سازی

هر سیستم‌عاملِ مهمان برای کار کردن فرض می‌کند که تنها مالک ماشین است و در حالت هسته اجرا می‌شود. طبق اسلایدها، سخت‌افزار در زمان بوت در حالت هسته شروع می‌کند و فقط در این حالت می‌توان دستورهای ممتاز (مثل تنظیم زمان‌سنج، مدیریت وقفه، کنترل I/O و پاک‌کردن حافظه) را اجرا کرد. بنابراین یک سیستم‌عامل مهمان از سخت‌افزار این انتظارها را دارد:

- **پردازنده:** اجرای آزاد دستورهای ممتاز، تنظیم زمان‌سنج و کنترل‌گر وقفه، ماسک‌کردن وقفه‌ها، تعویض جدول وقفه (IDT) و داشتن CPU به‌صورت انحصاری. زمان‌بندی را خودش انجام می‌دهد و هیچ دلیلی ندارد پردازنده را به سیستم‌عامل دیگری بدهد.
- **حافظه:** کل حافظهٔ فیزیکی (از آدرس صفر و پیوسته) را از آنِ خود می‌داند، جدول صفحه و MMU را خودش مقداردهی می‌کند و می‌تواند هر آدرس فیزیکی را بخواند یا بنویسد.
- **ورودی/خروجی:** دسترسی مستقیم به ثبات‌های کنترل‌گر دستگاه‌ها، درگاه‌ها، خطوط وقفه و DMA؛ و انتظار دارد دستگاه‌ها دقیقاً در همان حالتی باشند که خودش رها کرده است.

اگر چند سیستم‌عامل هم‌زمان چنین کنترلی داشته باشند، مشکلات زیر پیش می‌آید:

- **تداخل حافظه:** هر هسته روی حافظهٔ دیگری می‌نویسد؛ یک سیستم‌عامل می‌تواند هستهٔ سیستم‌عامل دیگر را پاک کند (همان سناریوی «برنامهٔ کاربر می‌تواند سیستم‌عامل را پاک کند» در اسلایدِ نبودِ حالت دوگانه، این بار میان سیستم‌عامل‌ها).
- **تداخل در I/O:** چند سیستم‌عامل هم‌زمان در یک دستگاه می‌نویسند (اسلاید: «چند برنامه می‌توانند هم‌زمان روی یک دستگاه بنویسند») و دستورهای متناقض به یک کنترل‌گر می‌رسد؛ معلوم هم نیست یک وقفه باید به کدام سیستم‌عامل تحویل شود.

- **تداخل در زمان‌بندی:** زمان‌سنج و کنترل‌گر وقفه یکی است و هر سیستم‌عامل آن را به شکل خودش تنظیم می‌کند؛ تنظیمِ دومی تنظیمِ اولی را خراب می‌کند. هیچ‌کس نمی‌تواند CPU را از سیستم‌عاملی که وقفه‌ها را غیرفعال کرده یا در حلقهٔ بی‌نهایت است پس بگیرد؛ تقسیم عادلانهٔ منابع ناممکن است.
- **اجرای نادرست:** همه انتظار دارند توسط میان‌افزار بوت شوند و جدول وقفه، جدول صفحه و ثبات‌های کنترلی را در اختیار بگیرند؛ در هر لحظه فقط یک مجموعه از این وضعیت‌ها روی سخت‌افزار می‌تواند فعال باشد و تعویض میان سیستم‌عامل‌ها (ذخیره و بازیابی کل وضعیت ماشین) کار هیچ‌کدام نیست.
- **امنیت و پایداری:** هیچ مرز حفاظتی وجود ندارد؛ خطا یا بدافزار در یک سیستم‌عامل کل ماشین را از کار می‌اندازد یا اطلاعات بقیه را می‌خواند.

چرا لایهٔ واسط لازم است؟ چون باید یک مرجع واحد وجود داشته باشد که کنترل واقعی منابع را در دست دارد و بقیه را به رعایت قواعد مجبور می‌کند. این لایه همان ناظر ماشین مجازی (Hypervisor/VMM) است؛ طبق اسلایدها نرم‌افزار (یا سخت‌افزار)ی است که ماشین‌های مجازی را می‌سازد و اجرا می‌کند (نوع ۱ مستقیماً روی سخت‌افزار) و برای هر مهمان این توهم را می‌سازد که کامپیوتر خصوصی خودش را دارد. وظایف جداسازی آن:

- **پردازنده:** زمان‌بندی vCPUها روی هسته‌های واقعی با زمان‌سنجی که فقط ناظر کنترل می‌کند (پیش‌دستی اجباری؛ به همان دلیل که سیستم‌عامل با زمان‌سنج از انحصار پردازه‌ها جلوگیری می‌کند).
- **حافظه:** نگاشت «فیزیکی مهمان → فیزیکی واقعی» با جدول صفحهٔ سایه یا جدول صفحهٔ تودرتو، تخصیص و سهمیه‌بندی حافظه و جلوگیری از دسترسی به حافظهٔ دیگران.
- **ورودی/خروجی:** ارائهٔ دستگاه‌های مجازی (شبیه‌سازی‌شده یا نیمه‌مجازی) یا تخصیص اختصاصی دستگاه همراه با IOMMU، و تحویل وقفه‌های مجازی به مهمان درست.
- **حفاظت از خود و مهار خطا:** دستورهای ممتاز فقط در دست ناظر است؛ خرابی یا آلودگی یک مهمان فقط خودش را از کار می‌اندازد و میزبان و مهمان‌های دیگر محافظت می‌شوند (اسلاید: «Host system protected from other VMs»).

منبع: اسلاید (ص‌های حالت دوگانه در فصل ۱؛ بخش ماشین‌های مجازی در فصل ۲)؛ جزئیات IOMMU و جدول صفحهٔ تودرتو تکمیلی.

بخش ب – اجرای هستهٔ مهمان مستقیماً در حالت هستهٔ واقعی

اگر ناظر اجازه دهد هستهٔ مهمان در حالت هستهٔ واقعی اجرا شود، عملاً ناظر کنترل ماشین را از دست داده است:

- **امنیت و دسترسی غیرمجاز:** مهمان می‌تواند هر دستور ممتازی را اجرا کند؛ پس می‌تواند هر آدرس فیزیکی (حافظهٔ ناظر و مهمان‌های دیگر) را بخواند و بنویسد، جدول صفحه و CR3 را عوض کند یا با DMA حافظهٔ دیگران را بخواند. هیچ سازوکاری نمی‌تواند دسترسی غیرمجاز را به تله بیندازد.
- **جداسازی و اثرگذاری روی دیگران:** مهمان می‌تواند جدول وقفه یا کد ناظر را بازنویسی کند و حتی ناظر را دور بزند. چون وقفه‌ها را ماسک و زمان‌سنج را دوباره‌تنظیم می‌کند، ناظر هرگز کنترل CPU را پس نمی‌گیرد؛ مهمانِ دارای حلقهٔ بی‌نهایت بقیه را گرسنه نگه می‌دارد (همان مسئلهٔ «process hogging» که زمان‌سنج در اسلایدها برای جلوگیری از آن آمده است).
- **کنترل و تخصیص منابع:** ناظر نمی‌تواند سهمیه بگذارد یا محدودیتی اعمال کند، چون مهمان خود را مالک تمام حافظه و تمام CPUها می‌داند. دستگاه‌های I/O به‌طور مستقیم و هم‌زمان توسط چند مهمان برنامه‌ریزی می‌شوند و وقفه‌ها و پیکربندی‌ها با هم تداخل می‌کنند؛ اشتراک‌گذاری کنترل‌شدهٔ دستگاه ناممکن است.
- **پایداری کل سیستم:** خطا در هستهٔ مهمان (اشارهگر خراب، دستور HLT، خطای سه‌گانه، پیکربندی غلط سخت‌افزار) مستقیماً ماشین واقعی را متوقف یا بازنشانی می‌کند و فقط به همان مهمان محدود نمی‌ماند. ناظر هم نمی‌تواند وضعیت مهمان را ببیند یا ذخیره کند، پس فعالیت‌هایی مثل عکس‌گیری (snapshot) و

انتقال زندهٔ ماشین مجازی هم ممکن نیست.

نتیجه: هستهٔ مهمان باید از حالت هستهٔ واقعی «تنزل» پیدا کند (مثلاً به حالت کاربر، مانند بخش بعد) یا سخت‌افزار حالت جداگانه‌ای برای مهمان فراهم کند (بخش ه).

منبع: اسلاید (حفاظت با حالت دوگانه و زمان‌سنج، فصل ۱؛ مزایای VM، فصل ۲)؛ تحلیل تفصیلی تکمیلی.

بخش ج – روش «تله و شبیه‌سازی» (Trap-and-Emulate)

روش کار. ناظر در حالت هستهٔ واقعی اجرا می‌شود و هستهٔ مهمان را در حالت کاربر واقعی اجرا می‌کند؛ مهمان فقط یک «حالت کاربر مجازی» و «حالت هستهٔ مجازی» دارد که هر دو روی حالت کاربر واقعی اجرا می‌شوند (همان‌طور که اسلایدِ پیاده‌سازی VM می‌گوید). مراحل:

- دستورهای معمولی مهمان، چه کد کاربر و چه کد هستهٔ مهمان، مستقیماً و با سرعت تقریباً کامل روی CPU اجرا می‌شوند.
- وقتی مهمان دستور ممتازی اجرا می‌کند (مثلاً تغییر جدول صفحه، غیرفعال‌کردن وقفه، دستور I/O)، چون CPU در حالت کاربر است، استثنا (تله) رخ می‌دهد و کنترل با ذخیرهٔ وضعیت به ناظر منتقل می‌شود.
- ناظر دستور را رمزگشایی می‌کند و اثر آن را روی وضعیت مجازی مهمان (ثبات‌ها و بیت حالت مجازی، جدول صفحهٔ مجازی، دستگاه مجازی) شبیه‌سازی می‌کند و نه روی سخت‌افزار واقعی.
- ناظر وضعیت مهمان را بازیابی می‌کند و اجرای مهمان را از دستور بعدی ادامه می‌دهد.

محدودیت‌ها در معماری‌هایی که برای مجازی‌سازی طراحی نشده‌اند.

- **دستورهای حساسِ غیرممتاز:** در برخی معماری‌ها (مثل x86 پیش از افزوده‌شدن پشتیبانی سخت‌افزاری) دستورهایی هستند که وضعیت یا پیکربندی سیستم را می‌خوانند یا تغییر می‌دهند ولی ممتاز نیستند؛ مثل `POPF` و `SGDT`. این دستورها در حالت کاربر تله ایجاد نمی‌کنند، پس ناظر هرگز مطلع نمی‌شود و نتیجهٔ نادرست به مهمان می‌رسد (تفصیل در بخش د).
- مجبور می‌شویم از راه‌حل‌های سنگین‌تر مانند بازنویسی دودویی (ترجمهٔ کد مهمان) یا نیمه‌مجازی‌سازی (تغییر دادن هستهٔ مهمان) استفاده کنیم.
- **هزینهٔ زیاد تله‌ها:** هستهٔ مهمان تعداد زیادی دستور ممتاز اجرا می‌کند و هر کدام به تله می‌افتد؛ این هم باعث افت کارایی می‌شود (اسلاید: اجرای هستهٔ مجازی در حالت کاربر واقعی «ممکن است به کارایی آسیب بزند و به پشتیبانی سخت‌افزاری نیاز دارد»).

چرا انتقال‌های زیاد کارایی را کم می‌کند؟ هر انتقال شامل ایجاد استثنا، ذخیرهٔ وضعیت مهمان، سوئیچ به زمینهٔ ناظر، رمزگشایی و شبیه‌سازی دستور، بازیابی وضعیت و بازگشت است. اگر دستور ممتاز روی ماشین واقعی حدود یک چرخهٔ ساعت طول می‌کشد، همین دستور اینجا صدها تا هزاران چرخه هزینه دارد. افزون بر آن، هر انتقال خط لولهٔ CPU را خالی می‌کند و کش و TLB را آلوده می‌کند. چون هستهٔ مهمان مدام جدول صفحه را به‌روز می‌کند، وقفه‌ها را فعال و غیرفعال می‌کند و به ثبات‌های دستگاه دسترسی می‌دهد (و هر عملیات I/O چندین دسترسی به ثبات دارد)، این هزینه چندین برابر می‌شود.

منبع: اسلاید (پیاده‌سازی VM، فصل ۲)؛ مثال‌های x86 و تحلیل هزینه تکمیلی.

بخش د – ناتوانی معماری در تله‌انداختن همهٔ رفتارهای حساس

بر اساس شرط پوپک–گلدبرگ، معماری‌ای برای مجازی‌سازی با تله و شبیه‌سازی مناسب است که مجموعهٔ دستورهای حساس (آن‌هایی که وضعیت و پیکربندی منابع را می‌خوانند یا عوض می‌کنند یا بسته به حالت رفتار متفاوت دارند) زیرمجموعهٔ دستورهای ممتاز باشد. اگر چنین نباشد:

- **ایجاد مشکل در پیاده‌سازی:** مهمان دستور حساس را در حالت کاربر مستقیماً اجرا می‌کند و ناظر خبردار نمی‌شود. یا دستور بی‌صدا نتیجهٔ اشتباه می‌دهد (مثلاً POPF در حالت کاربر تغییر پرچم وقفه را نادیده می‌گیرد، در حالی که مهمان فکر می‌کند وقفه‌ها را غیرفعال کرده است و منطق هسته‌اش خراب می‌شود)، یا اطلاعات واقعی ماشین را به مهمان نشان می‌دهد (مثلاً SGDT آدرس واقعی جدول توصیف‌گر سراسری را برمی‌گرداند و نه آدرس مجازی). در نتیجه ناظر دیگر نمی‌تواند به مهمان توهم «یک ماشین کامل» بدهد (ویژگی هم‌ارزی).
- **نقض جداسازی:** اگر اثر دستور روی حالت واقعی سخت‌افزار برود، یک مهمان بر وضعیت مشترک ماشین اثر می‌گذارد و بقیهٔ ماشین‌های مجازی و حتی ناظر را تحت تأثیر قرار می‌دهد. اگر دستور اطلاعات بخواند، اطلاعات ناظر یا مهمان‌های دیگر نشت می‌کند و مهمان می‌فهمد روی ماشین مجازی اجرا می‌شود. در هر دو حالت، ناظر کنترل کامل منابع را ندارد (ویژگی کنترل منابع) و مرز حفاظتی شکسته می‌شود.
- **چرا تشخیص صحیح عملیات حساس نیازمندی اصلی است؟** چون هر سه ویژگی مورد نیاز مجازی‌سازی، یعنی وفاداری (مهمان همان رفتار ماشین واقعی را ببیند)، کنترل کامل منابع توسط ناظر و کارایی (بیشتر دستورها مستقیم اجرا شوند)، وقتی برقرار است که ناظر دقیقاً در لحظه‌های لازم دخالت کند. تشخیص ناقص یعنی یا ناظر دخالت نمی‌کند (خطای رفتار و نقض جداسازی)، یا مجبور است همه‌چیز را شبیه‌سازی کند (کارایی ضعیف).

منبع: اسلاید (لزوم پشتیبانی سخت‌افزاری برای VM، فصل ۲)؛ شرط پوپک–گلدبرگ و مثال‌های دستورها تکمیلی.

بخش ه – پشتیبانی سخت‌افزاری از مجازی‌سازی

پردازنده‌های جدید (مثل Intel VT-x و AMD-V) یک حالت اجرای جدید اضافه می‌کنند که اسلایدها «حالت میزبان/ناظر» در برابر «حالت مهمان» می‌نامند (مثال اسلاید: AMD). ناظر در حالت میزبان اجرا می‌شود و مهمان در حالت مهمان؛ هستهٔ مهمان در حالت مهمان، حالت هستهٔ مجازی خودش را دارد و می‌تواند بیشتر دستورهای ممتاز را مستقیماً اجرا کند. هر رویدادی که ناظر باید در آن دخالت کند (مثلاً دسترسی به دستگاه، تغییر بعضی ثبات‌های کنترلی، خطای صفحهٔ تودرتو) با تنظیمات ناظر باعث «خروج از ماشین مجازی» (VM exit) می‌شود و CPU به‌طور خودکار وضعیت مهمان را در ساختار سخت‌افزاری (VMCS/VMCB) ذخیره و وضعیت ناظر را بارگذاری می‌کند.

تأثیر بر طراحی و عملکرد ناظر:

- **ساده‌تر شدن طراحی:** ناظر دیگر لازم نیست کد مهمان را اسکن یا بازنویسی کند و کد هسته‌ی مهمان را هم تغییر نمی‌دهد (بدون بازنویسی دودویی و بدون نیمه‌مجازی‌سازی). ناظر به یک حلقهٔ رویدادمحور تبدیل می‌شود: ورود به مهمان، خروج با علت مشخص، پردازش علت (شبیه‌سازی دستگاه یا مدیریت منابع) و ورود دوباره.
- چون همهٔ رفتارهای حساس دیگر قابل‌تله‌انداختن‌اند، مشکل دستورهای حساسِ غیرممتاز از بین می‌رود.
- ذخیره و بازیابی وضعیت مهمان را سخت‌افزار انجام می‌دهد و ناظر مجبور نیست آن را دستی ساخته و نگه دارد.
- جدول صفحهٔ تودرتو (EPT/NPT) ترجمهٔ آدرس مهمان را به سخت‌افزار می‌سپارد و ناظر از نگهداری جدول صفحهٔ سایه خلاص می‌شود.

- **کارایی:** مهمان‌های بدون تغییر با سرعت نزدیک به بومی اجرا می‌شوند و فقط رویدادهای واقعاً حساس هزینهٔ خروج را می‌پردازند.

منبع: اسلاید (حالت میزبان/مهمان، فصل ۲)؛ جزئیات VMCS، EPT تکمیلی.

۲ وقفه به‌عنوان ابزار حمله

بخش اول: حملهٔ Heckler

بخش آ – چرا گردانندهٔ فراخوانی سیستمی در اجرای ناخواسته به‌صورت وقفه خطرناک است؟

تله و وقفه از یک جدول بردار و یک مسیر مشترک ورود به هسته استفاده می‌کنند (اسلاید: وقفه کنترل را از طریق بردار وقفه به روال سرویس منتقل می‌کند و تله یا فراخوانی سیستمی یک وقفهٔ نرم‌افزاری است). گردانندهٔ فراخوانی سیستمی بر چند فرض بنا شده است:

- کسی با اجرای عمدی دستور تله (مثل `int 0x80`) آن را صدا زده است و نشانی بازگشت، حالت و ثبات‌ها بر اساس قرارداد فراخوانی آماده شده‌اند.
- بر اساس اسلاید «جریان فراخوانی سیستمی»، هسته «دستور وقفه‌دهنده» را بررسی می‌کند تا نوع را بفهمد و سپس پارامترهای ثبات را کنترل و فراخوانی را اجرا می‌کند؛ یعنی منبع و نیت فراخوانی را مفروض می‌گیرد.
- فراخواننده یک پردازهٔ کاربر است که خودش از اثرهای فراخوانی (فقط نتیجه در یک ثبات) آگاه است.

وقتی همین گردانندهٔ همین بردار با یک وقفهٔ تزریقی، یعنی ناهم‌زمان و در یک لحظهٔ دلخواه، اجرا شود، هیچ‌کدام از این فرض‌ها برقرار نیست. سخت‌افزار برای وقفه‌های خارجی (و تزریق‌شده) بررسی سطح دسترسی دروازه را انجام نمی‌دهد (این بررسی فقط برای دستور نرم‌افزاری `int n` است)، و مسیر مشترک هم نمی‌تواند تشخیص دهد که وقفه از یک دستور واقعی آمده است یا از بیرون. در نتیجه هسته محتوای تصادفی ثبات‌های پردازهٔ قربانی را به‌عنوان شمارهٔ فراخوانی و آرگومان می‌پذیرد، یک فراخوانی سیستمی دلخواه را با اختیارات پردازه اجرا می‌کند و نتیجه را روی ثبات‌هایی می‌نویسد که پردازه در حال استفاده از آن‌هاست. مهاجم زمان تزریق را طوری انتخاب می‌کند که اجرای پردازه را به نفع خود تغییر دهد.

منبع: اسلاید (وقفه، تله و جریان فراخوانی سیستمی، فصل ۱)؛ جزئیات سطح دسترسی دروازه و ماهیت حملهٔ Heckler تکمیلی.

بخش ب – اهمیت مقدار ثبات‌ها در لحظهٔ حمله

هسته شمارهٔ فراخوانی را از ثبات `rax` و آرگومان‌ها را از ثبات‌های `rdi`، `rsi`، `rdx`، `r10`، `r8` و `r9` می‌خواند و مقدار برگشتی را هم در `rax` می‌گذارد (اسلاید: اطلاعات اضافی ممکن است در فایل ثبات یا پشته باشد). نتیجه:

- مقدار ثبات‌ها در لحظهٔ تزریق از نظر مهاجم «ورودیِ فراخوانی» است. مهاجم با انتخاب لحظه می‌تواند شمارهٔ فراخوانی و آرگومان‌های مطلوب را بسازد، چون ثبات‌ها محتوای محاسبه‌ای را دارند که پردازه در همان لحظه انجام می‌دهد.
- نتیجهٔ فراخوانی ساختگی روی `rax` می‌نشیند. اگر `rax` در آن لحظه مقدار برگشتی تابعی مثل بررسی گذرواژه باشد (تابع در `rax` نتیجه می‌گذارد)، مقدار برگشتی با نتیجهٔ فراخوانی ساختگی (مثلاً صفر یعنی موفقیت) جایگزین می‌شود و شاخهٔ برنامه عوض می‌شود. روش دور زدن احراز هویت در sudo و OpenSSH همین است.
- چون سیستم‌عامل مهمان محرمانه است، ناظر نمی‌تواند حافظه و ثبات‌ها را ببیند. او حالت ثبات‌ها را با زمان‌بندی (مثلاً شمارش وقفه‌ها یا رویدادها پیش از ورود به تابع حساس) حدس می‌زند و بنابراین حمله به زمان‌بندی دقیق و مقدار ثبات‌ها وابسته است.

منبع: تکمیلی (مقالهٔ Heckler، USENIX Security 2024؛ قرارداد ثبات‌ها در x86-64).

بخش ج – ویژگی روال رسیدگی امن و یک راه دفاع

روالی در هر لحظه بی‌خطر است که (۱) وضعیت ناشی از اجرای خودش را به‌طور کامل ذخیره و بازیابی کند و هیچ اثری روی برنامهٔ وقفه‌خورده نگذارد (در اسلایدها: سیستم‌عامل ثبات‌ها و شمارندهٔ برنامه را ذخیره می‌کند)؛ (۲) فرضی دربارهٔ محتوای ثبات‌ها یا زمان فراخوانی نداشته باشد و ورودی‌هایش را اعتبارسنجی کند («Kernel verifies passed parameters»)؛ (۳) تنها از مسیر و منبع مجاز قابل ورود باشد؛ یعنی بداند به‌دلیل یک دستور واقعی در حالت کاربر فراخوانده شده است؛ (۴) بازورودپذیر باشد و با ماسک‌کردن مناسب وقفه‌ها به وضعیت ناسازگار نرسد.

راه دفاع پیشنهادی: چند لایه ممکن است:

- **سطح هسته:** اگر مسیر `int 0x80` (فراخوانی سیستمی قدیمی ۳۲بیتی) لازم نیست، غیرفعال شود (برای مثال با تنظیمات `ia32_emulation`) تا بردار 0x80 گردانندهٔ فعال نداشته باشد. لینوکس برای مهمان‌های Intel TDX و AMD SEV این قابلیت را به‌صورت پیش‌فرض غیرفعال می‌کند.
- **سطح سخت‌افزار و ناظر:** محدودکردن بردارهایی که ناظر اجازه دارد تزریق کند (مثل ویژگی تزریق محدود/جایگزین در AMD SEV-SNP و بررسی تزریق در Intel TDX)، تا بردارهای ویژه‌ای مثل 0x80 و بردارهای استثنا قابل‌تزریق نباشند.
- **سطح گرداننده:** اجرای گردانندهٔ فراخوانی سیستمی فقط وقتی مجاز باشد که با دستور واقعی در حالت کاربر آمده باشد (برای نمونه ثبت منشأ ورود در مسیر مشترک و رد وقفه‌های بدون منشأ معتبر).

منبع: تکمیلی؛ مفهوم بازورودی و ذخیرهٔ وضعیت در اسلاید.

بخش دوم: نشت اطلاعات از طریق وقفه‌ها (حملهٔ شمارنده در مرورگر)

بخش د – زبانه‌ها چگونه از هم باخبر می‌شوند و چرا هر وب‌گاه الگوی خودش را دارد؟

دو زبانه حافظهٔ مشترک ندارند، ولی **سخت‌افزار و هسته را به اشتراک گذاشته‌اند**. زبانهٔ مهاجم حلقه‌ای می‌چرخاند که یک شمارنده را زیاد می‌کند و در هر بازهٔ زمانی کوتاه (مثلاً چند میلی‌ثانیه) مقدار آن را ثبت می‌کند. هر وقت CPU (یا همان هستهٔ پردازنده‌ای که حلقه روی آن است) به کار دیگری مشغول شود، شمارنده کمتر زیاد می‌شود و در دنباله نقطهٔ افتادگی می‌ماند. دو عامل اصلی این افتادگی‌ها هستند:

- **وقفه‌ها:** هر وقفه (کارت شبکه، پردازندهٔ گرافیکی، دیسک، زمان‌سنج، وقفهٔ بین‌هسته‌ای) CPU را متوقف می‌کند تا گرداننده اجرا شود؛ اسلاید می‌گوید پردازنده با هر وقفه وضعیت را ذخیره می‌کند و به گردانندهٔ مربوط می‌رود. مدت و زمان این وقفه‌ها از رفتار «زبانهٔ قربانی» می‌آید.
- **رقابت بر منابع مشترک:** کش، پهنای باند حافظه، منابع داخلی هسته و فرکانس پردازنده.

هر وب‌گاه الگوی خاص خودش را می‌سازد، چون اندازه، تعداد و ترتیب منابع و درخواست‌های شبکه، زمان‌بندی دریافت بسته‌ها، رندر و اجرای جاوااسکریپت (و در نتیجه فعالیت GPU و CPU) برای هر وب‌گاه متفاوت است. نتیجه یک «اثر انگشت» زمانی از وقفه‌ها و بار CPU است که شمارنده آن را به‌صورت یک سری زمانی ثبت می‌کند و مدل یادگیری ماشین (مثل شبکهٔ عصبی کانولوشنی) آن را به نام وب‌گاه نگاشت می‌کند.

منبع: اسلاید (وقفه و پیامدهای آن برای CPU، فصل ۱)؛ تحلیل حمله تکمیلی (مقالهٔ There's Always a Bigger Fish).

بخش ه – سنجش سه روش دفاعی

۱) فرستادن وقفه‌های دستگاه به یک هستهٔ دیگر. وقفه‌های دستگاه‌ها (شبکه، GPU، دیسک) دیگر هستهٔ مهاجم را متوقف نمی‌کنند؛ پس بزرگ‌ترین بخش سیگنال از بین می‌رود. اما کامل نیست: وقفهٔ زمان‌سنج و وقفه‌های بین‌هسته‌ای (در ادامه) باقی می‌مانند؛ کار نرم‌افزاری مرتبط با وقفه‌ها (مثلاً پردازش بسته‌ها) ممکن است هنوز روی هستهٔ مهاجم اجرا شود؛ و کانال‌های دیگری مثل کش و پهنای باند حافظه باقی‌اند. هزینه: یک هستهٔ پرمشغله برای همهٔ وقفه‌ها (افت کارایی و تأخیر) و نیاز به چسباندن پردازه‌ها به هسته. **میزان جلوگیری: متوسط (کاهش قابل توجه ولی نه حذف کامل).**

۲) کم‌دقت و نوفه‌دار کردن ساعت. حمله به ساعت دقیق نیاز ندارد: شمارنده فقط تعداد تکرار حلقه در یک بازهٔ چندمیلی‌ثانیه‌ای را می‌شمارد و وقفه‌ها (از چند میکروثانیه تا میلی‌ثانیه) در این مقیاس ظاهر می‌شوند. یعنی ساعتی با دقت چند ده یا صد میکروثانیه هم هنوز همین کار را می‌کند. مهاجم می‌تواند ساعت جایگزین هم بسازد (با نخ شمارنده و SharedArrayBuffer). این روش فقط وقتی اثر دارد که ساعت چنان خراب شود که مرورگر عملاً بی‌فایده شود. **میزان جلوگیری: کم (ضعیف).** این روش بیشتر در برابر حمله‌هایی کارساز است که به ساعت دقیق نیاز دارند.

۳) تزریق وقفه‌های ساختگی و تصادفی. وقفه‌های اضافه، الگوی واقعی را با نوفه می‌پوشانند و دقت دسته‌بند را کم می‌کنند. اما نوفهٔ مستقل از قربانی را می‌توان با میانگین‌گیری از ردهای تکراری و فیلتر کردن تا حدی حذف کرد. برای اینکه الگو واقعاً پنهان شود، نوفه باید هم‌اندازهٔ سیگنال و از نظر شکل و مدت شبیه وقفه‌های واقعی باشد. هزینه: هدر دادن CPU و انرژی و افزایش تأخیر. **میزان جلوگیری: متوسط، با هزینهٔ زیاد.**

روش	مهم‌ترین نقطهٔ قوت	مهم‌ترین نقطهٔ ضعف	میزان جلوگیری
۱) انتقال وقفه‌ها	حذف وقفه‌های دستگاه‌ها	زمان‌سنج، بین‌هسته‌ای، کش	متوسط
۲) ساعت کم‌دقت	ساده و ارزان	حمله به ساعت دقیق نیاز ندارد	کم
۳) وقفهٔ ساختگی	پنهان‌سازی الگوی واقعی	میانگین‌گیری و هزینهٔ زیاد	متوسط

چرا وقفهٔ زمان‌سنج و وقفه‌های بین‌هسته‌ای را نمی‌توان به هستهٔ دیگر منتقل کرد؟

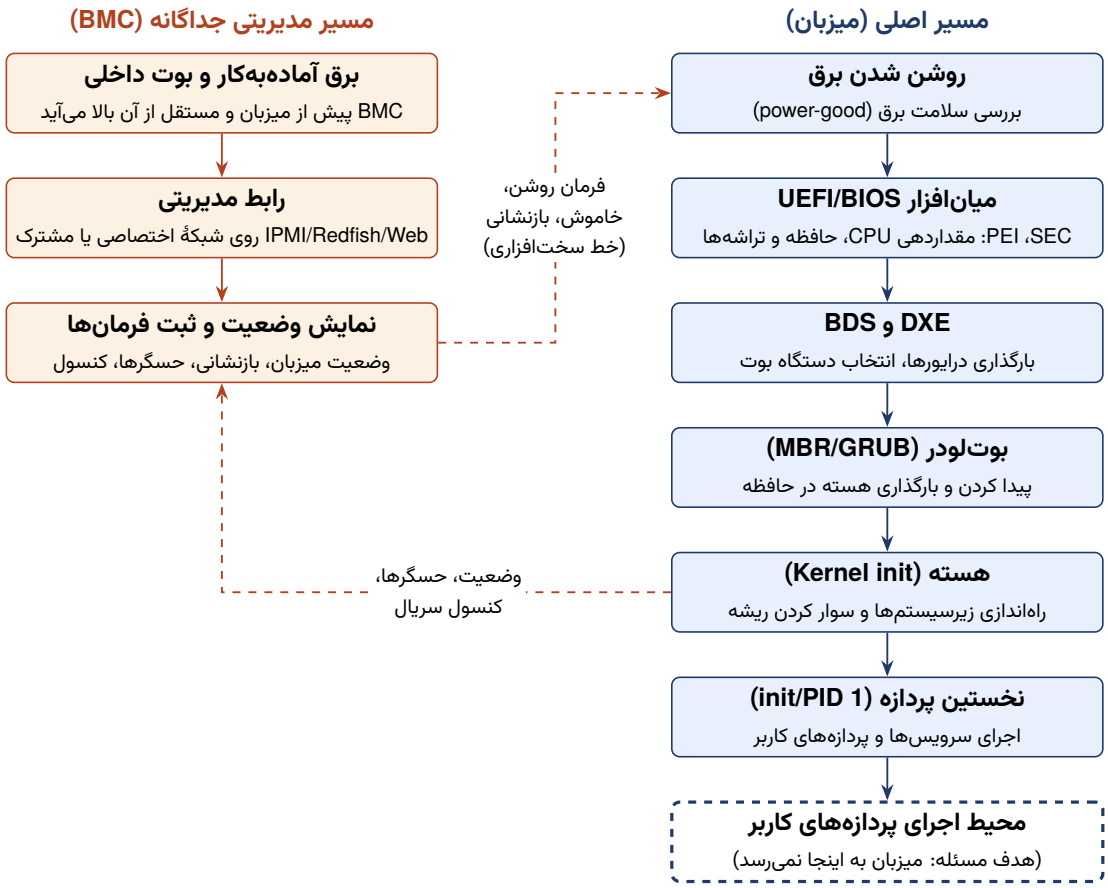
- **وقفهٔ زمان‌سنج:** هر هسته زمان‌سنج محلی خودش را دارد (مثلاً LAPIC timer) و سیستم‌عامل برای زمان‌بندی روی همان هسته به آن نیاز دارد. طبق اسلایدها، سیستم‌عامل پیش از اجرای پردازه زمان‌سنج را تنظیم می‌کند تا پس از پایان سهم زمانی کنترل را پس بگیرد. اگر این وقفه به هستهٔ دیگر برود، هستهٔ اجراکنندهٔ پردازه هرگز پیش‌دستی نمی‌شود و سهم زمانی و زمان‌بندی از بین می‌رود.
- **وقفه‌های بین‌هسته‌ای (IPI):** ماهیتاً پیامی از یک هسته به هستهٔ هدف مشخص است تا حالت همان هسته عوض شود؛ مثل ابطال TLB همان هسته، بیدار کردن یا پیش‌دستی روی آن، یا اجرای یک تابع روی آن. منتقل کردن آن به هسته‌ای دیگر معنایش را از بین می‌برد. در مقابل، وقفهٔ دستگاه از کنترل‌گر وقفه به هر هسته‌ای قابل مسیریابی است.

منبع: اسلاید (زمان‌سنج، فصل ۱)؛ بقیهٔ تحلیل تکمیلی.

۳ کنترل‌گر مدیریت برد (BMC) و راه‌اندازی

بخش آ – مسیر راه‌اندازی میزبان و جایگاه BMC

طبق اسلایدها: برنامهٔ راه‌اندازی (میان‌افزار، در ROM/EPROM) هنگام روشن شدن یا راه‌اندازی دوباره اجرا می‌شود، همه‌چیز را مقداردهی می‌کند (ثبات‌های CPU، کنترل‌گرهای دستگاه)، هسته را بارگذاری می‌کند و هسته نخستین پردازه را اجرا می‌کند؛ مثال لینوکس: BIOS → MBR bootloader → GRUB → Kernel init → INIT → Run level. اسلاید UEFI هم مراحل SEC، PEI، DXE، BDS را نشان می‌دهد. BMC رایانهٔ کوچکی (معمولاً SoC مبتنی بر ARM) روی برد اصلی است که **در مسیری جدا** قرار دارد.



توضیح تصویر: مسیر اصلی (راست) از برق، میان‌افزار، بوت‌لودر و هسته به نخستین پردازه می‌رسد. BMC (چپ) رایانهٔ مستقل با برق آماده‌به‌کار، میان‌افزار و رابط شبکهٔ خودش است و فقط از طریق خطوط سخت‌افزاری و پیوندهای جانبی (روشن/بازنشانی، حسگرها، کنسول سریال) با مسیر میزبان ارتباط دارد. به همین دلیل در تمام مراحل و حتی وقتی میزبان به محیط اجرای پردازه‌های کاربر نمی‌رسد، فعال می‌ماند.

منبع: اسلاید (فصل ۱: راه‌اندازی رایانه، BMC، BIOS to OS Initialization).

بخش ب – آیا رابط مدیریتی BMC پردازه‌ای در سیستم‌عامل میزبان است؟

خیر. دلیل‌ها:

- BMC یک رایانهٔ کوچک مستقل روی برد اصلی است (اسلاید: «Small computer that sits on every server MB»، معمولاً ARM-based SoC) با CPU، حافظه و میان‌افزار (و معمولاً سیستم‌عامل) خودش. پردازه در مفهوم درس یعنی برنامه‌ای که هستهٔ میزبان آن را ایجاد و زمان‌بندی می‌کند و از حافظهٔ میزبان استفاده می‌کند؛ رابط BMC هیچ‌کدام را ندارد.

- برق آماده‌به‌کار و درگاه شبکهٔ اختصاصی (یا مشترک از طریق NC-SI) دارد؛ پس حتی وقتی میزبان خاموش است یا هنوز هسته‌ای بارگذاری نشده، کار می‌کند. چیزی که پیش از وجود سیستم‌عامل کار می‌کند نمی‌تواند پردازهٔ همان سیستم‌عامل باشد.
- بر اساس صورت مسئله، رابط مدیریتی با مسیر مدیریتی جداگانه از میزبان است و حتی وقتی سیستم‌عامل میزبان اجرا نشده پاسخ می‌دهد. بنابراین نه از هستهٔ میزبان زمان می‌گیرد و نه به آن وابسته است.

بخش ج – آیا پاسخ‌گو بودن BMC در حالی که میزبان به محیط پردازه‌ها نرسیده تناقض ایجاد می‌کند؟

خیر، تناقضی نیست، چون BMC و میزبان دو سامانهٔ جدا هستند.

- **آنچه می‌توان نتیجه گرفت:** برق آماده‌به‌کار برقرار است، BMC بوت شده و مسیر مدیریتی (شبکه و رابط) سالم است، و BMC می‌تواند وضعیت سخت‌افزاری برد را بخواند (برق، حسگرها) و فرمان بازنشانی را ثبت کند.
- **آنچه نمی‌توان نتیجه گرفت:** BMC اطلاعات خود را از سیگنال‌ها و حسگرهای سخت‌افزاری می‌گیرد؛ مثلاً می‌بیند میزبان «روشن است». این به معنای سلامت یا اجرای میان‌افزار، بوت‌لودر، هسته یا هر پردازه‌ای نیست. نمی‌توان نتیجه گرفت که میزبان به‌درستی بوت شده یا از کار نیفتاده است؛ فقط می‌توان گفت میزبان به محیط اجرای پردازه‌های کاربر نرسیده است و علت آن (میان‌افزار، بوت‌لودر، هسته یا سخت‌افزار) را باید جای دیگر پیدا کرد.

بخش د – بارگذاری هسته پس از ثبت درخواست بازنشانی

پس از روشن شدن دوباره، ابتدا میان‌افزار (UEFI/BIOS) اجرا می‌شود و سخت‌افزار را مقداردهی می‌کند، سپس بوت‌لودر (GRUB) **هسته** را از دیسک در حافظه بارگذاری می‌کند و کنترل را به آن می‌دهد (اسلاید: برنامهٔ راه‌اندازی هسته را بارگذاری می‌کند و هسته نخستین پردازه را شروع می‌کند). **ثبت درخواست بازنشانی به معنای اجرای سیستم‌عامل میزبان توسط BMC نیست**، زیرا:

- BMC فقط یک «فرمان» ثبت و ارسال می‌کند (فعال‌کردن خط بازنشانی یا چرخهٔ برق؛ معادل زدن دکمهٔ reset) و کد یا حافظهٔ میزبان را اجرا نمی‌کند.
- پس از دریافت سیگنال، مسئولیت اجرا با سخت‌افزار و میان‌افزار میزبان است که از ابتدا شروع می‌کنند و فرایند بوت را طی می‌کنند. کد BMC روی پردازندهٔ جداگانه‌ای اجرا می‌شود و اصلاً به میزبان وارد نمی‌شود.
- هسته‌ای که بارگذاری می‌شود همان است که روی دیسک میزبان قرار دارد و BMC در انتخاب یا بارگذاری آن نقشی ندارد (مگر فرمان‌هایی مثل تعیین دستگاه بوت در دفعهٔ بعد، که باز هم فقط یک تنظیم برای میان‌افزار است).

بخش ه – میزبان به درخواست‌های پردازهٔ کاربر پاسخ می‌دهد، ولی رابط BMC در دسترس نیست

نمی‌توان نتیجه گرفت که سیستم‌عامل میزبان از کار افتاده است. سیستم‌عامل میزبان به BMC وابسته نیست؛ پاسخ‌گو بودن پردازه‌های کاربر نشان می‌دهد که هسته، زمان‌بندی و مسیر شبکهٔ میزبان سالم است. نبودن رابط BMC به یکی از این علت‌ها می‌تواند باشد: BMC در حال راه‌اندازی دوباره یا به‌روزرسانی میان‌افزار است، خود BMC (میان‌افزار یا سخت‌افزارش) خراب شده، شبکهٔ مدیریتی یا درگاه اختصاصی (یا مسیر NC-SI) قطع است، یا تنظیمات دسترسی عوض شده است. یعنی خرابی BMC فقط توانایی مدیریت از راه دور را کم می‌کند و نشانهٔ خرابی میزبان نیست.

بخش و – مقایسهٔ دو حالت

حالت	مسیری که کار می‌کند	مسیری که ممکن است از کار افتاده باشد
BMC در دسترس، سیستم‌عامل میزبان در دسترس نیست	مسیر برون‌باند BMC: دیدن وضعیت برق و حسگرها، روشن/خاموش‌کردن و بازنشانی، کنسول سریال از راه دور، ثبت رویدادها	مسیر درون‌باند میزبان: هسته، پردازه‌های کاربر، سرویس‌هایی مثل SSH، شبکهٔ میزبان؛ (سیستم‌عامل پایین است یا اصلاً بالا نیامده)
BMC در دسترس نیست، سیستم‌عامل میزبان در دسترس است	مسیر درون‌باند میزبان: سیستم‌عامل، پردازه‌ها و شبکهٔ میزبان؛ (به‌جز حالتی که کارت شبکه را مشترک با BMC استفاده کنند و آن مسیر هم قطع شود)	مسیر مدیریتی برون‌باند: رابط BMC، کنترل از راه دور برق و بازنشانی، کنسول، حسگرها؛ (برق و میزبان مشکلی ندارند)

منبع: اسلاید (راه‌اندازی رایانه و BMC، فصل ۱)؛ استدلال دربارهٔ مسیرهای درون‌باند و برون‌باند و NC-SI تکمیلی.

۴ سیگنال‌ها در لینوکس

بخش آ – رفتارهای پیش‌فرض پس از ورود سیگنال

برای هر سیگنال فقط پنج رفتار پیش‌فرض ممکن است (راهنمای `signal(7)`):

- `Term`: پایان‌دادن به پردازه (مثل `SIGTERM`، `SIGINT`، `SIGKILL`، `SIGALRM`).
- `Core`: پایان‌دادن به پردازه و ساختن فایل core dump از حافظهٔ پردازه (مثل `SIGSEGV`، `SIGABRT`، `SIGFPE`).
- `Ign`: نادیده‌گرفتن سیگنال (مثل `SIGCHLD`، `SIGURG`).
- `Stop`: متوقف‌کردن پردازه (مثل `SIGSTOP`، `SIGTSTP`).
- `Cont`: ادامه‌دادن پردازهٔ متوقف‌شده (`SIGCONT`).

دو مورد را توضیح می‌دهیم:

- `Term` **(مثلاً** `SIGTERM`**، شمارهٔ ۱۵):** هسته پردازه را بدون ذخیرهٔ وضعیت متوقف می‌کند و منابعش را (حافظه، فایل‌های باز) آزاد می‌کند. والد با `wait` وضعیت پایان را می‌بیند. اگر برنامه گردانندهٔ سیگنال نصب کرده باشد، به‌جای رفتار پیش‌فرض آن اجرا می‌شود (برای مثال برای بستن مرتب فایل‌ها). `SIGKILL` و `SIGSTOP` را نمی‌توان گرفت، مسدود یا نادیده گرفت.
- `Core` **(مثلاً** `SIGSEGV`**، شمارهٔ ۱۱):** وقتی پردازه به آدرس نامعتبر دسترسی می‌دهد، سخت‌افزار تله ایجاد می‌کند، هسته آن را به سیگنال تبدیل می‌کند و پردازه پایان می‌یابد و تصویر حافظه و ثبات‌ها در فایل core dump برای اشکال‌زدایی ذخیره می‌شود (اسلاید فصل ۲: «Failure of an application can generate core dump file»).

منبع: تکمیلی (`signal(7)`)، اشاره به core dump و تله در اسلایدها.

بخش ب – مؤلفه‌های مرتبط با سیگنال در `/proc/PID/status`

این فایل چند فیلد دربارهٔ سیگنال‌ها دارد که همگی **نقاب بیتی شانزده‌دهی** هستند: بیت شمارهٔ n-1 روشن یعنی سیگنال شمارهٔ n در آن مجموعه است. سه مؤلفهٔ اصلی:

- `SigBlk` (Blocked): سیگنال‌های **مسدودشده** (نقاب سیگنال). تحویل سیگنال‌های این مجموعه به تعویق می‌افتد تا نقاب برداشته شود. مثال: `SigBlk: 0000000000010000` یعنی بیت ۱۶ (سیگنال ۱۷، یعنی `SIGCHLD`) مسدود است.
- `SigIgn` (Ignored): سیگنال‌هایی که پردازه **نادیده می‌گیرد** (رفتار `SIG_IGN`).
- `SigCgt` (Caught): سیگنال‌هایی که پردازه برایشان **گردانندهٔ سفارشی** ثبت کرده است (مثل `sigHandler()` در شکل ۱ صورت تمرین).

فیلدهای مرتبط دیگر: `SigPnd` و `ShdPnd` (سیگنال‌های **در انتظار** برای نخ و برای کل پردازه) و `SigQ` (تعداد سیگنال‌های در صف نسبت به سقف مجاز، مثلاً `0/31538`).

منبع: تکمیلی (`proc(5)`).

بخش ج – سیگنال‌های تله‌های فراخوانی نامعتبر و خطای ممیز شناور

- **فراخوانی سیستمی نامعتبر:** `SIGSYS` (شمارهٔ ۳۱ در x86-64، «Bad system call»)، رفتار پیش‌فرض `Core`. نکته: در لینوکس یک شمارهٔ فراخوانی ناموجود معمولاً فقط خطای `ENOSYS` برمی‌گرداند و سیگنال نمی‌فرستد. `SIGSYS` وقتی ارسال می‌شود که یک فیلتر seccomp فراخوانی را رد کند (`SECCOMP_RET_TRAP`) یا در سیستم‌های یونیکس دیگر مثل BSD و Solaris.
- **خطا در محاسبات (از جمله شناور):** `SIGFPE` (شمارهٔ ۸، «Floating-point exception»)، رفتار پیش‌فرض `Core`. با وجود نامش، علاوه بر خطاهای ممیز شناور (تقسیم بر صفر، سرریز و ...) برای تقسیم صحیح بر صفر یا سرریز صحیح نیز ارسال می‌شود. سازوکار آن همان تله است که اسلایدها توضیح می‌دهند (تقسیم بر صفر یک وقفهٔ نرم‌افزاری ناشی از خطاست): CPU استثنا ایجاد می‌کند، هسته آن را به `SIGFPE` تبدیل و به پردازه ارسال می‌کند. نوع خطا از `si_code` (مثل `FPE_INTDIV`، `FPE_FLTDIV`) مشخص می‌شود.

منبع: تکمیلی (`signal(7)`، `sigaction(2)`)؛ مفهوم تله در اسلاید.

بخش د – چرا امکان ماسک‌کردن وقفه لازم است؟

- **حفاظت از بخش‌های بحرانی و جلوگیری از شرایط رقابتی:** اگر گرداننده در میانهٔ به‌روزرسانی یک ساختمان داده (مثلاً صف زمان‌بندی) اجرا شود و همان ساختمان داده را تغییر دهد، داده ناسازگار می‌شود. با ماسک‌کردن، اجرای اتمیک تضمین می‌شود (برای نمونه هسته هنگام تغییر ساختمان‌های مشترک با گرداننده وقفه را غیرفعال می‌کند، و برنامه با `sigprocmask` سیگنال‌ها را مسدود می‌کند).
- **جلوگیری از تودرتوشدن و بازورود:** هنگام اجرای گرداننده، همان وقفه (یا وقفه‌های با اولویت پایین‌تر) نباید دوباره آن را قطع کند؛ وگرنه سرریز پشته یا ناسازگاری وضعیت رخ می‌دهد. سیستم‌عامل ذخیره و بازیابی وضعیت را هم باید بدون مزاحمت انجام دهد (اسلاید: سیستم‌عامل ثبات‌ها و شمارندهٔ برنامه را ذخیره می‌کند).
- **مرحلهٔ راه‌اندازی و تغییر حالت:** هنگام بوت، پیش از تنظیم جدول وقفه و گردانندها، یا هنگام سوئیچ زمینه، دریافت وقفه خطرناک است.
- **اولویت‌بندی و کنترل بار:** وقفه‌های مهم‌تر روی کم‌اهمیت‌ها مقدم می‌شوند و طوفان وقفه (بار زیاد) را می‌توان مهار کرد. وقفه ازدست‌نرفته است و تا برداشتن نقاب در حالت انتظار می‌ماند (در سیگنال‌های استاندارد چند

ارسال پشت‌سرهم یکی حساب می‌شوند). وقفه‌های حیاتی (مثل خطای سخت‌افزاری شدید) غیرقابل‌ماسک (NMI) هستند تا هرگز نادیده گرفته نشوند.

منبع: تکمیلی؛ اشاره به ذخیرهٔ وضعیت هنگام وقفه در اسلاید فصل ۱.